Sequencing — report with images example

Local rendering of live OpenProject API data. NOT a native OpenProject export. Native routes returned HTTP 406.

Scope: #215–#220 · All six native statuses are preserved. No dates or completion evidence invented.

Contents

#215 — Sequencing delivery waves — A → B1 → C (dependency map)

#216 — Wave A — Sequencing domain, serializers, typed IDs and structure hashes

#217 — Wave B1 — database-neutral repository seam

#218 — Wave C3 — scoring validation memo and formula objective

#219 — Wave C4 — ranking and resolver (ST-022 candidate comparator)

#220 — Wave C5 — fixtures parity and Wave C exit proof

Images option: no embedded image references were found in the selected descriptions. No illustrative image has been invented.

#215 — Sequencing delivery waves — A → B1 → C (dependency map)

Native state: New · Type: Epic · Priority: Normal

Source description (including source claims; implementation not independently verified)

Parent for the Sequencing delivery waves. Created 2026-10-28 because "Wave A / B1 / C" were three undefined terms inside an agent instruction file with no tracked dependency structure — which is what a tracker is for, not an instruction file.

Scope read from `docs/orchestration/context/02_MESO_WAVE_CONTRACTS.md` and `docs/orchestration/WAVE-C_resolver_and_scientific_values.md`, not invented.

Wave	Delivers	State
A	Sequencing domain: types, serializers, typed IDs, invariants, structure hashes, canonical fixtures (`lib/features/sequencing/domain`)	landed on `master`
B1	Database-neutral repository seam	landed on `master` (three B1 packets)
C	Resolver and scientific values; deterministic feasible candidates, policy-versioned projections, ranked resolution	partly landed, C3–C5 open

Wave C depends on A + B1. Cloud persistence is NOT a prerequisite — that is SSOT-D-059 territory and deliberately deferred.

Wave C exit proof, verbatim: "RES-01–RES-18 and SCI-01–SCI-09 pass." Plus: all claimed IDs have executable tests, generation is deterministic, and Wave A/B1 hashes and APIs remain stable.

The standing invariant this structure protects: a storage decision never changes the domain layer. Wave A types, serializers, IDs and structure hashes are storage-neutral. That rule stays in the repo rule file; the *work* lives here.

Caveat carried from the source: these wave contracts are dependency/provenance maps and were marked not-executable while the SSOT-materialization Micro is active. Confirm before scheduling C4/C5.

Sequencing screen design and execution package — 2026-11-03

Status: screen contracts and materialization are written and pushed; the screens are not yet implemented in the app. This section is the durable execution map. Repository snapshot: [`d9965b39`](https://github.com/leela-spec/Leela-Cloud-2026/tree/d9965b39ae0e0a493b8acb6752f92dc19a37df7c).

Authority and design rule

For this screen-design task, resolve evidence in this order: **screen contracts → current code → operator-authored fixture data → older SSOT narrative**. Domain ownership remains with the domain artifacts. Do not let a screen invent, flatten, rerank, mutate, or recalculate owner facts.

Owned screen contracts

- [Family owner: inventory, artifact ledger, 12 conflicts, 10 open aspects, critical path](https://github.com/leela-spec/Leela-Cloud-2026/blob/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/docs/ssot/screens/sequencing-family.md)
- [Recommendation Feed](https://github.com/leela-spec/Leela-Cloud-2026/blob/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/docs/ssot/screens/sequencing-recommendations.md)
- [Manual Selection](https://github.com/leela-spec/Leela-Cloud-2026/blob/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/docs/ssot/screens/sequencing-manual-selection.md)
- [Details](https://github.com/leela-spec/Leela-Cloud-2026/blob/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/docs/ssot/screens/sequencing-details.md)
- [Modification](https://github.com/leela-spec/Leela-Cloud-2026/blob/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/docs/ssot/screens/sequencing-modification.md)
- [Run](https://github.com/leela-spec/Leela-Cloud-2026/blob/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/docs/ssot/screens/sequencing-run.md)
- [Finished](https://github.com/leela-spec/Leela-Cloud-2026/blob/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/docs/ssot/screens/sequencing-finished.md)
- [Structure Gallery](https://github.com/leela-spec/Leela-Cloud-2026/blob/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/docs/

ssot/screens/sequencing-structure-gallery.md)
- [Screen contract index](https://github.com/leela-spec/Leela-Cloud-2026/blob/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/docs/ssot/screens/index.md)
- [Sequencing materialization rows](https://github.com/leela-spec/Leela-Cloud-2026/blob/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/docs/ssot/features/sequencing/materialization.csv#L181-L187)

End-to-end flow

`Canonical fixtures → shared presentation projection → Structure Gallery verification → Recommendations/Details → Accept or Modify → Run → Finished`

1. Structure Gallery — first visual checkpoint

- **Input:** canonical fixtures, `LeelaSequenceEnvelope`, current `LeelaChunkDisplay`, structural relations, points and bindings.
- **Process:** add `/lab/sequencing/structures`; load fixtures through `FixtureSequencingRepository`; select a fixture; render real shared widgets; show IDs/facts; exercise narrow/wide, text scale, reduced motion and failure states.
- **Output:** one read-only lab screen where every canonical structure can be checked before production screens reuse it.
- **Example:** selecting `inst_parallel_audio_001` must show both simultaneous occurrences, their roles/bindings and shared timing without converting them into a fake sequence.

2. Shared presentation projection — one interpretation

- **Input:** `SequenceInstance`, `SlotOccurrence`, `ExecutionBinding`, `CompositeRelation`, `ParallelLayer`, Chunk facts and TP/XP/BP projections.
- **Process:** preserve IDs; resolve each occurrence and binding; create typed display models; nest relation groups; feed the envelope and Chunk display; fail visibly on inconsistent identity instead of inventing data.
- **Output:** one projection reused by Gallery, Recommendations, Details, Modification, Run and Finished.
- **Example:** a Bundle with three occurrences becomes one Bundle presentation containing three independently identifiable Chunk presentations, not three unrelated icons.

3. Recommendations and Details — first production vertical slice

- **Input:** owner-ranked complete candidates plus classification, duration, points, evidence and explicit favorite state.
- **Process:** preserve owner ordering; render the shared projection; open a candidate or accepted Instance by typed ID; distinguish candidate actions from immutable Instance actions; never accept or start implicitly.
- **Output:** a read-only browse-and-inspect path.
- **Example:** a 25-minute Focus Session shows TP/XP/BP, occurrence/support structure and evidence, then exposes `Accept`, `Modify` or `Back`; an accepted Instance exposes `Start Run` or `Create modified copy`.

4. Manual Selection, acceptance and Modification

- **Input:** Mode, subtype, scope, context, timing constraints, a baseline candidate/Instance, supported edit commands and validation rules.
- **Process:** send intent to the resolution owner; receive a complete candidate; keep the baseline immutable; edit a transient proposal; revalidate after each edit; calculate exact signed TP/XP/BP deltas; accept only a complete valid proposal.
- **Output:** a new accepted Instance with its own typed ID and structure hash; the baseline remains unchanged.
- **Example:** removing an optional occurrence may change 30→24 minutes, TP 40→33, XP 18→16 and BP 4→4; the UI shows `−6`, `−7`, `−2`, `0`.

5. Run — factual execution

- **Input:** accepted Instance, valid `runId`, exact occurrences/relations/bindings, lifecycle timestamps and persisted Run events.
- **Process:** restore Run state; derive the active occurrence from facts; render the real Chunk display; derive time from timestamps/monotonic deadline; append start/pause/resume/complete/skip/substitution events; never rewrite the accepted Instance.
- **Output:** resumable execution with an append-only factual history.
- **Example:** substituting Chunk B for accepted Chunk A appends a deviation containing `occurrenceId`, accepted Chunk, actual Chunk and reason; it does not edit the Instance.

6. Finished — accepted versus actual

- **Input:** accepted projection, closed Run, realized timeline, deviations and actualizer-produced TP/XP/BP.
- **Process:** compare accepted and actual occurrences; show completed/skipped/shortened/substituted facts; show projected

versus actual duration and points; hand longitudinal history to Stats.
- **Output:** an auditable completion summary.
- **Example:** accepted 30 min / TP40 / XP18 / BP4 versus actual 24 min / TP32 / XP16 / BP4, with substitution and shortening reasons. Never pro-rate rewards from elapsed percentage.

7. Route convergence and legacy retirement

- **Input:** verified new screens, current routes/prototypes, compatibility needs and navigation tests.
- **Process:** add typed routes; point normal navigation to them; redirect legacy routes when needed; reject empty or timestamp-generated fake Instance IDs; prove workflow parity before archiving old screens.
- **Output:** `Recommendations → Details → Accept or Modify → Run → Finished`.
- **Example:** replace `/sequences/run` with empty Instance ID by `/sequences/runs/:runId`, which loads a real Run, its accepted Instance and event history.

Settled conflicts that implementation must not reopen

1. Favorites means explicit saved/starred state, not “most done”.
2. Display `Free Time`; normalize legacy `Recharge` only on read.
3. Display `Mix It Up`; wire code remains `mixItUp`.
4. Start requires valid accepted Instance and Run IDs; no placeholders/timestamps.
5. Actual TP/XP/BP comes from the versioned actualizer, not elapsed-percentage proration.
6. Preserve occurrences, repetitions, roles, Bundle, Fusion, Parallel Layers and timing; never flatten to icon lists.
7. Finished centers accepted projection versus actual Run; account totals belong to Stats.
8. BP means Balance Points, never Bonus Points.
9. Every `LeelaChunkDisplay` receives owner-published facts for its exact binding.
10. Run timing derives from persisted timestamps or a monotonic deadline and reconciles after lifecycle resume.
11. Substitution appends a deviation event and preserves the accepted Instance.
12. Platform/activity strips are metadata; typed relations and `LeelaSequenceEnvelope` carry structure.

Open aspects and safe defaults

1. Separate Play Mode landing versus direct Manual Selection: do not build an extra route first.
2. Dominant-versus-Mixed threshold: show per-occurrence symbols or `Mixed`; do not invent a threshold.
3. Notifications by platform: Run must remain correct without notifications.
4. Auto-advance versus confirmation: show declared ending policy; do not infer it from duration.
5. Prev/Next semantics: no silent completion; state changes append events.
6. Substitution alternatives: show only owner-returned feasible choices and reasons.
7. Sharing Finished: no functional Share CTA until Community/privacy owns it.
8. Feedback persistence: no saveable form without an owner-backed command/schema.
9. Celebration: one short motion-gated confirmation; reduced motion shows final state immediately.
10. Gallery release status: keep it lab-only and read-only until explicitly promoted.

Grounding artifacts

- [Canonical sequencing fixtures](https://github.com/leela-spec/Leela-Cloud-2026/tree/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/assets/mock/sequencing)
- [Fixture repository](https://github.com/leela-spec/Leela-Cloud-2026/blob/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/lib/features/sequencing/data/repositories/fixture_sequencing_repository.dart)
- [Sequence envelope](https://github.com/leela-spec/Leela-Cloud-2026/blob/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/lib/design_system/components/leela_sequence_envelope.dart)
- [Chunk display](https://github.com/leela-spec/Leela-Cloud-2026/blob/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/lib/design_system/components/leela_chunk_display.dart)
- [Current Run prototype](https://github.com/leela-spec/Leela-Cloud-2026/blob/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/lib/pages/scr_sequence_run.dart)
- [Current Finished prototype](https://github.com/leela-spec/Leela-Cloud-2026/blob/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/lib/pages/scr_sequence_finished.dart)
- [Current harmonized Run prototype](https://github.com/leela-spec/Leela-Cloud-2026/blob/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/lib/features/harmonization/harmonized_run_screen.dart)
- [Current builder prototype](https://github.com/leela-spec/Leela-Cloud-2026/blob/d9965b39ae0e0a493b8acb6752f92dc19a37df7c/lib/features/harmonization/harmonized_sequence_builder_screen.dart)
- [Historical screen-spec source directory](https://github.com/leela-spec/Leela-Cloud-2026/tree/d9965b39ae0e0a493b8acb6752f

92dc19a37df7c/docs/reference-corpus/spec_repo/LeelaAppDevelopment/StatusQuo/Sequencing)

Verification and delivery order

First deliver the Structure Gallery plus shared projection. Then deliver the read-only Recommendations/Details slice. Add acceptance/Modification next. Add factual Run events and lifecycle timing after that. Build Finished from closed-Run facts. Converge routes and retire legacy prototypes last.

Repository evidence: screen contracts commit `acbbad39`; materialization commit `d9965b39`; `python scripts/gates.py --since origin/master` reported **all 5 executed gates passed, 10 correctly parked** at delivery time.

#216 — Wave A — Sequencing domain, serializers, typed IDs and structure hashes

Native state: New · Type: Task · Priority: Normal

Source description (including source claims; implementation not independently verified)

Landed on `master`. Tracked for dependency provenance, not as open work.

Delivers the Sequencing domain layer: types, serializers, typed IDs, invariants, structure hashes and canonical fixtures under `lib/features/sequencing/domain/`.

The invariant it establishes, which every later wave must preserve: a storage decision never changes the domain layer. Wave A types, serializers, IDs and structure hashes are storage-neutral. Preserve the Wave A structure-hash projection exactly unless the Canon changes it explicitly — this sentence has been lost from the repo instruction surface twice, once for 57 minutes.

Canonical fixture manifest entries are `{path, sha256}` and the hash is verified before decoding.

Rule home: `.claude/rules/sequencing.md` (SSOT-D-060). Spec: `docs/ssot/features/sequencing/spec.md`.

Observed relations in the common example scope

#88: #217 follows #216

#217 — Wave B1 — database-neutral repository seam

Native state: New · Type: Task · Priority: Normal

Source description (including source claims; implementation not independently verified)

Landed on `master` via three B1 implementation packets. Tracked for dependency provenance.

Delivers the repository seam that lets the domain run without committing to a storage engine. Pairs with SSOT-D-059: fixture-first and database-neutral, cloud persistence deferred.

Depends on: Wave A.

Blocks: Wave C. `WAVE-C_resolver_and_scientific_values.md` states the dependency as "Wave A (domain) + Wave B1 (database-neutral repositories). Cloud persistence is not a prerequisite."

Observed relations in the common example scope

#89: #218 follows #217

#88: #217 follows #216

#218 — Wave C3 — scoring validation memo and formula objective

Native state: New · Type: Task · Priority: Normal

Source description (including source claims; implementation not independently verified)

OPEN. The part of C3 that did not land.

SSOT-D-031's extraction (session packets 10–13) landed real Algorithm/Sequencing capability on `master` — `lib/features/algorithm/**` and `lib/features/sequencing/{application,domain/scoring}/**` — which substantially satisfies C1/C2 and *part* of C3's scoring-contract scaffolding.

What remains, verbatim from the source: "no XP/TP/BP formula arithmetic was ever on the branch, so C3's actual validation-memo/formula objective remains open."

Depends on: Wave A + Wave B1 (both landed).

Observed relations in the common example scope

#90: #219 follows #218

#89: #218 follows #217

#219 — Wave C4 — ranking and resolver (ST-022 candidate comparator)

Native state: New · Type: Task · Priority: Normal

Source description (including source claims; implementation not independently verified)

BLOCKED. Packet `docs/orchestration/wave-c/04-ranking-and-resolver.md`, revised in place to cite the real landed files.

Delivers the `ST-022` candidate comparator: deterministic ranked resolution over Algorithm-produced feasible candidates.

Prohibited donor inputs, explicitly: XP/min ranking, flat slots, equal-duration division, and old harmonization authority.

Depends on: Wave C3.

Observed relations in the common example scope

#91: #220 follows #219

#90: #219 follows #218

#220 — Wave C5 — fixtures parity and Wave C exit proof

Native state: New · Type: Task · Priority: Normal

Source description (including source claims; implementation not independently verified)

BLOCKED. Packet `docs/orchestration/wave-c/05-fixtures-parity-and-exit.md`.

Delivers generated fixtures plus fixture/session parity proof, and carries the Wave C exit gate.

Exit proof, verbatim: "RES-01–RES-18 and SCI-01–SCI-09 pass." Plus: every claimed ID has an executable test, generation is deterministic, and Wave A/B1 hashes and APIs remain stable.

Depends on: Wave C4.

Observed relations in the common example scope

#91: #220 follows #219